

1. 실습 목적

이번 실습의 목적은 CMake와 OpenCV를 이용하여 영상 처리 프로그램을 작성하고 실행하는 것이다.

컬러로 된 원본 영상을 불러온 후 그레이 영상으로 변환하고, 변환된 그레이 영상을 다시 이진 영상으로 변환하여 각각의 결과를 화면에 출력한다.

실습 과정은 다음과 같다.

원본 컬러 영상 → 그레이 영상 → 이진 영상

2. 소스 파일

2-1 main.cpp

```
GNU nano 7.2 src/main.cpp
#include <opencv2/opencv.hpp>
#include <iostream>

using namespace cv;
using namespace std;

int main()
{
    // 원본 이미지 불러오기
    Mat image = imread("lena.jpg");

    // 이미지가 제대로 불러왔는지 확인
    if (image.empty())
    {
        cout << "이미지를 불러올 수 없습니다." << endl;
        return -1;
    }

    // 그레이 영상
    Mat gray;
    cvtColor(image, gray, COLOR_BGR2GRAY);

    // 이진 영상
    Mat binary;
    threshold(gray, binary, 128, 255, THRESH_BINARY);

    // 결과 출력
    imshow("Original Image", image);
    imshow("Gray Image", gray);
    imshow("Binary Image", binary);

    // 키 입력을 기다림
    waitKey(0);

    return 0;
}
```

2-2 CMakeLists.txt

```
GNU nano 7.2 CMakeLists.txt
cmake_minimum_required(VERSION 3.10)

project(ImageProject)

find_package(OpenCV REQUIRED)

add_executable(image_program src/main.cpp)

target_link_libraries(image_program ${OpenCV_LIBS})
```

3. main.cpp 라인 단위 코드 설명

Mat image = imread("lena.jpg");

lena.jpg 파일을 읽어서 image라는 변수에 저장한다.

Mat은 OpenCV에서 이미지를 저장할 때 사용하는 자료형이다.

쉽게 말하면 image라는 상자에 원본 사진을 넣는 과정이다.

if (image.empty())

이미지가 제대로 불러와졌는지 확인한다.

image.empty()가 참이면 이미지가 정상적으로 불러와지지 않았다는 뜻이다.

cout << "이미지를 불러올 수 없습니다." << endl;

이미지를 불러오지 못했을 경우 터미널에 오류 메시지를 출력한다.

return -1;

이미지를 불러오지 못했기 때문에 프로그램을 종료한다.

Mat gray;

그레이 영상을 저장할 gray 변수를 만든다.

아직 영상이 들어 있지 않은 빈 공간을 준비하는 것이다.

cvtColor(image, gray, COLOR_BGR2GRAY);

원본 컬러 영상 image를 그레이 영상으로 변환하여 gray에 저장한다.

COLOR_BGR2GRAY는 컬러 영상을 회색조 영상으로 변환하라는 의미이다.

즉,

컬러 영상 → 흑백(회색조) 영상

으로 변환하는 부분이다.

Mat binary;

이진 영상을 저장할 binary 변수를 만든다.

threshold(gray, binary, 128, 255, THRESH_BINARY);

그레이 영상을 이진 영상으로 변환한다.

여기서 128은 밝기를 구분하는 기준값이다.

픽셀의 밝기가 128보다 크면 255(흰색), 128 이하이면 0(검은색)으로 변환한다.

따라서 영상이 검은색과 흰색으로 구분된다.

쉽게 표현하면 다음과 같다.

밝기 128 이하 → 검은색

밝기 128 초과 → 흰색

imshow("Original Image", image);

원본 컬러 영상을 화면에 출력한다.
창의 제목은 Original Image로 표시된다.

imshow("Gray Image", gray);
변환된 그레이 영상을 화면에 출력한다.
창의 제목은 Gray Image로 표시된다.

imshow("Binary Image", binary);
변환된 이진 영상을 화면에 출력한다.
창의 제목은 Binary Image로 표시된다.

waitKey(0);
사용자가 키보드의 아무 키나 누를 때까지 프로그램을 기다리게 한다.
이 명령어가 있기 때문에 출력된 이미지 창이 바로 닫히지 않는다.

return 0;
프로그램이 정상적으로 실행되었음을 나타내고 프로그램을 종료한다.

4. CMakeLists.txt 라인 단위 설명

cmake_minimum_required(VERSION 3.10)
프로그램을 빌드할 때 필요한 CMake의 최소 버전을 지정한다.
여기서는 CMake 3.10 이상을 사용하도록 설정하였다.

project(ImageProject)
CMake 프로젝트의 이름을 ImageProject로 설정한다.

find_package(OpenCV REQUIRED)
컴퓨터에 설치되어 있는 OpenCV를 찾는다.
REQUIRED는 OpenCV가 반드시 필요하다는 의미이다.

add_executable(image_program src/main.cpp)
src/main.cpp 소스코드를 이용하여 image_program이라는 실행 프로그램을 만들도록 설정한다.

target_link_libraries(image_program \${OpenCV_LIBS})
image_program에서 OpenCV 라이브러리를 사용할 수 있도록 연결한다.
즉, main.cpp에서 사용한 imread(), cvtColor(), imshow() 등의 OpenCV 기능을 실제 프로그램과 연결하는 역할을 한다.

5. 프로그램 빌드 및 실행

먼저 프로젝트 폴더로 이동

cd ~/image_project

CMake를 이용하여 빌드에 필요한 파일을 생성

cmake -S . -B build

이 과정에서 CMake는 CMakeLists.txt를 읽고 프로그램을 빌드하는 데 필요한 설정 파일을 생성한다.

다음 명령어를 이용하여 프로그램을 빌드

cmake --build build

빌드가 정상적으로 완료되면 image_program 실행 파일이 생성된다.

다음 명령어를 사용하여 프로그램을 실행

./build/image_program

7. 실행 결과 분석

이번 실습에서는 OpenCV를 이용하여 하나의 컬러 영상을 그레이 영상과 이진 영상으로 변환하였다.

원본 영상은 여러 가지 색상 정보를 가지고 있지만, 그레이 영상으로 변환하면 색상 정보가 제거되고 밝기 정보만 남는다.

이후 threshold 함수를 이용하여 밝기 128을 기준으로 픽셀을 검은색과 흰색으로 구분하였다.

이를 통해 원본 컬러 영상 → 그레이 영상 → 이진 영상으로 영상이 변환되는 과정을 확인할 수 있었다.

또한 CMake를 이용하여 OpenCV 라이브러리와 C++ 소스 파일을 연결하고 프로그램을 빌드하는 방법을 확인할 수 있었다.

8. 실행 과정

```
linux@BOOK-M8LSJ7JPPD:~$ cd ~
linux@BOOK-M8LSJ7JPPD:~$ pwd
/home/linux
linux@BOOK-M8LSJ7JPPD:~$ pkg-config --modversion opencv4
4.6.0
linux@BOOK-M8LSJ7JPPD:~$ cd ~
linux@BOOK-M8LSJ7JPPD:~$ mkdir image_project
linux@BOOK-M8LSJ7JPPD:~$ cd image_project
linux@BOOK-M8LSJ7JPPD:~/image_project$ mkdir src
linux@BOOK-M8LSJ7JPPD:~/image_project$ nano src/main.cpp
linux@BOOK-M8LSJ7JPPD:~/image_project$ nano CMakeLists.txt
linux@BOOK-M8LSJ7JPPD:~/image_project$ cmake -S . -B build
-- The C compiler identification is GNU 13.3.0
-- The CXX compiler identification is GNU 13.3.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Found OpenCV: /usr (found version "4.6.0")
-- Configuring done (4.8s)
-- Generating done (0.0s)
-- Build files have been written to: /home/linux/image_project/build
linux@BOOK-M8LSJ7JPPD:~/image_project$ ls build
CMakeCache.txt  CMakeFiles  Makefile  cmake_install.cmake
```

```
linux@BOOK-M8LSJ7JPPD:~/image_project$ cmake --build build
[ 50%] Building CXX object CMakeFiles/image_program.dir/src/main.cpp.o
[100%] Linking CXX executable image_program
[100%] Built target image_program
```

```
linux@BOOK-M8LSJ7JPPD:~/image_project$ pwd
/home/linux/image_project
```

```
linux@BOOK-M8LSJ7JPPD:~/image_project$ ./build/image_program
```

